Strategic Assessment: Constitutional Filing Doctrine & Lower Case Standardization

1. Executive Summary: The Constitutional Doctrine

This report analyzes the transition to a **Constitutional Filing Doctrine**. This strategy establishes a strict bi-level casing standard to distinguish the "Immutable Law" (Constitution) from the "Operational Machinery" (Execution).

- **The Doctrine:**
  - **Uppercased Canon (The Constitution):** Reserved strictly for the root governance files that define the repository's existence, history, and active contract. These files represent "Singular Truth."
  - **Lowercased Operations (The Machine):** All other files—code, manuals, manifests, registries, and definitions—must use lowercase (kebab-case or snake_case). This aligns with POSIX standards and treats them as utilitarian components rather than legal authority.
- **The Driver:**
  The operator observed that renaming .tpl files to lowercase currently results in lowercase outputs (or should), signaling a natural technical inclination towards lowercase standardization. However, an audit of ops/bin/compile reveals it currently *forces* uppercase output. Adopting this doctrine requires refactoring the compiler to align with the lowercase strategy.

---

2. Classification of Surfaces

Under the Constitutional Doctrine, the file system is partitioned into two distinct jurisdictional tiers.

2.1 Tier 1: The Constitution (Uppercase Preserved)

These files remain uppercase to denote their status as the "Source of Truth" or "Navigation Anchors." They are the only exceptions.

File	Function	Justification
PoT.md	**Law**	The Policy of Truth. The single source of

		governance.
SoP.md	**History**	The State of Play. The immutable ledger of changes.
PoW.md	**Proof**	The Proof of Work. The ledger of evidence.
TASK.md	**Contract**	The Active Surface. The dashboard for current work.
STELA.md	**Identity**	Project definitions (e.g., ops/lib/project/STELA.md).
README.md	**Anchor**	Standard git entry point (root and subdirectories).
INDEX.md	**Map**	Curated navigation indexes.

2.2 Tier 2: The Operations (Transition to Lowercase)

All other surfaces are demoted to lowercase to signify they are *functional* rather than *constitutional*.

- **Manifests:** ops/lib/manifests/CORE.md $\rightarrow$ core.md. (These are config files, not law).
- **Manuals:** docs/MANUAL.md $\rightarrow$ manual.md. (Instructions, not contract).
- **Maps:** docs/MAP.md $\rightarrow$ map.md. (Generated artifact).
- **Registries:** docs/ops/registry/AGENTS.md $\rightarrow$ agents.md. (Database lists).
- **Definitions:** opt/_factory/agents/R-AGENT-01.md $\rightarrow$ r-agent-01.md.

---

3. Technical Forensics: The Template Engine Constraint

You noted that *renaming .tpl files to lowercase causes the rendered file to become lowercase*.

Our code audit of the current ops/bin/compile binary reveals a critical friction point that must be addressed to implement this doctrine.

3.1 The Compiler Logic

Current logic in ops/bin/compile:

Bash

```
# [1]: ops/bin/compile
output_name="$(printf '%s' "$output_name" | tr '[:lower:]' '[:upper:]')"
```

Currently, the compiler **aggressively forces** uppercase output, regardless of the template's filename. If you rename core.md.tpl, the script will still attempt to output CORE.md unless patched.

3.2 The Fix: Passive Compilation

To support the Constitutional Doctrine, ops/bin/compile must be patched to respect the casing of the template file (or force lowercase), effectively decoupling the filename from the "Uppercase = Truth" legacy rule.

Required Patch:

Bash

```
# New logic: Output name matches template stem (lowercase)
local output_name="${template_file%.tpl}"
# output_name remains "core.md"
```

---

4. Impact Analysis & Broken Windows

Transitioning the "Operational Tier" to lowercase will immediately break specific hardcoded paths in the binaries and governance files. These must be remediated in the same commit as the rename.

4.1 Critical Breakage: The Context Engine

The ops/bin/context and ops/bin/llms binaries are hardcoded to look for uppercase manifests.

- **Failure:** ops/bin/context searches for ops/lib/manifests/OPS.md.
- **Outcome:** "File not found" errors on startup.
- **Remediation:** Update ops/lib/scripts/synthesize.sh and ops/bin/context to default to ops.md and core.md.

4.2 Critical Breakage: The Factory Guard

The tools/lint/factory.sh script validates that files in opt/_factory/ match their registry entries in docs/ops/registry/.

- **Failure:** Registry says R-AGENT-01; file system has r-agent-01.md.
- **Outcome:** CI pipeline (repo_gates.yml) fails, blocking merges.
- **Remediation:** Update linter to normalize IDs to lowercase before checking file existence (e.g., check for ${id,,}.md).

4.3 Governance Drift (PoT.md)

The Constitution itself (PoT.md) references docs/MANUAL.md and docs/MAP.md as Canon Surfaces.

- **Failure:** The Constitution points to non-existent (uppercase) files.
- **Outcome:** "Canon Drift" (System Failure State).
- **Remediation:** Amend PoT.md Section 1.3 to reference docs/manual.md and docs/map.md.

---

5. Strategic Options

Option A: The "Constitutional Doctrine" (Recommended)

Enforce Uppercase for Constitution, Lowercase for Operations. Patch tooling to support this split.

- **Pros:**
  - **Clear Hierarchy:** Visual distinction between "Law" (UPPER) and "machinery" (lower).
  - **Platform Standard:** Aligns with standard linux/devops casing for config and scripts.
  - **Technical Truth:** Acknowledges that manifests and manuals are utilitarian files, not sovereign laws.
- **Cons:**
  - **Transition Cost:** Requires simultaneous patching of ops/bin/compile, context, map, and lint.
- **Risk:** High (Tooling breakage if not patched atomically).

Option B: The "Total Lowercase" (Purist)

Lowercase *everything*, including pot.md and task.md.

- **Pros:** Absolute consistency. Zero ambiguity.
- **Cons:** Loss of "Authority" signaling. The PoT looks like just another markdown file.
- **Risk:** High (Cultural/Governance impact).

Option C: The "Status Quo" (Semantic Casing)

Keep current "Uppercase = Important" rule.

- **Pros:** Zero effort.
- **Cons:** High technical debt. Continued cross-platform case-sensitivity collisions (Windows/Linux).
- **Risk:** Low (immediate), High (long-term friction).

---

6. Execution Plan: The "Constitutional" Migration

To execute **Option A**, follow this sequence to avoid system lockout.

1. **Prep Tooling (The Patch):**
   - Modify ops/bin/compile to stop forcing uppercase.
   - Update ops/bin/context to look for ops.md.
   - Update tools/lint/factory.sh to handle lowercase file targets.
2. **Rename Operations:**
   - Rename ops/lib/manifests/*.md $\rightarrow$ lowercase.
   - Rename docs/*.md (except INDEX) $\rightarrow$ lowercase.
   - Rename opt/_factory/**/*.md $\rightarrow$ lowercase.
3. **Amend Constitution:**
   - Update PoT.md references to reflect new paths.
   - Update README.md links.
4. **Verify:**
   - Run ./ops/bin/context to ensure the AI can still read reality.

Recommendation

Adopt the Constitutional Doctrine.

It resolves the cross-platform technical debt of mixed casing while preserving the "Totem" status of the Constitutional files (PoT, TASK, SoP). The cost of patching ops/bin/compile is negligible compared to the long-term stability of a lowercase operational file system.